IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received August 2, 2021, accepted August 24, 2021, date of publication August 30, 2021, date of current version September 13, 2021.

Digital Object Identifier 10.1109/ACCESS.2021.3109081

IoT Intrusion Detection System Using Deep Learning and Enhanced Transient Search Optimization

ABDULAZIZ FATANI[1,2], MOHAMED ABD ELAZIZ[3], ABDELGHANI DAHOU[4], MOHAMMED A. A. AL-QANESS[5], AND SONGFENG LU[6,7]

[1]School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China
[2]Computer Science Department, Umm Al-Qura University, Makkah 24381, Saudi Arabia
[3]Department of Mathematics, Faculty of Science, Zagazig University, Zagazig 44519, Egypt
[4]LDDI Laboratory, Faculty of Science and Technology, University of Ahmed Draia, Adrar 01000, Algeria
[5]State Key Laboratory for Information Engineering in Surveying, Mapping and Remote Sensing, Wuhan University, Wuhan 430079, China
[6]Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China
[7]Shenzhen Huazhong University of Science and Technology Research Institute, Shenzhen 518057, China

Corresponding author: Songfeng Lu (lusongfeng@hust.edu.cn)

This work was supported in part by Hubei Provincial Science and Technology Major Project of China under Grant 2020AEA011, in part by the Key Research and Development Plan of Hubei Province of China under Grant 2020BAB100, and in part by the Project of Science, Technology and Innovation Commission of Shenzhen Municipality of China under Grant JCYJ20210324120002006.

ABSTRACT The great advancements in communication, cloud computing, and the internet of things (IoT) have opened critical challenges in security. With these developments, cyberattacks are also rapidly growing since the current security mechanisms do not provide efficient solutions. Recently, various artificial intelligence (AI) based solutions have been proposed for different security applications, including intrusion detection. In this paper, we propose an efficient AI-based mechanism for intrusion detection systems (IDS) in IoT systems. We leverage the advancements of deep learnings and metaheuristics (MH) algorithms that approved their efficiency in solving complex engineering problems. We propose a feature extraction method using the convolutional neural networks (CNNs) to extract relevant features. Also, we develop a new feature selection method using a new variant of the transient search optimization (TSO) algorithm, called TSODE, using the operators of differential evolution (DE) algorithm. The proposed TSODE uses the DE to improve the process of balancing between exploitation and exploration phases. Furthermore, we use three public datasets, KDDCup-99, NSL-KDD, BoT-IoT, and CICIDS-2017 to assess the performance of the developed method, which achieved higher accuracy compared to several existing approaches.

INDEX TERMS Internet of Things (IoT), security, cyberattack, intrusion detection system, feature selection, optimization algorithms.

I. INTRODUCTION

The Internet of Things (IoT) has ushered in a modern era in which a network of computers and devices capable of interacting and engaging with one another is propelling new business process technologies [1]. People and companies have experienced a broad range of issues related to credibility, enforcement, financing, and business operations as a result of widespread and rapid increase cybersecurity attacks on IoT systems [2]. Cloud computing can be defined as the model that supplies different services and resources to users on-demand, with minimal intervention between providers and users [3]. It has received significant attention among users and organizations. it is a part of IOT that stores IOT data. However, the transition process to cloud platforms is a complex problem due to the existence of different operations and security mechanisms. One of the most critical issues in cloud computing technology is security because of the huge amount of data storage in the cloud. The growth of cyberattacks is increased due to several reasons. The availability and easy access of hacking tools are one of the most important reasons since a hacker does not need comprehensive knowledge or brilliant skills to perform an attack [4].

With sufficient computing power and huge volume of data collected from interconnected devices, DL models can be considered to optimize the IoT security in terms of intrusion detection, user behaviors analysis, vulnerabilities, and privacy preserving. DL techniques and especially CNNs can

The associate editor coordinating the review of this manuscript and approving it for publication was M. Anwar Hossain.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

be used to learn, extract, and identify complex features and patterns directly from raw IoT data thus improving the utility of the device to efficiently detect possible threats and attacks in the IoT environment. Moreover, DL models are more efficient in automatic feature extraction rather than relying on traditional machine learning methods that demand handcrafted statistical features. In the past decades, researchers had developed many mechanisms for Intrusion Detection Systems [5]. Different machine learning (ML) techniques were proposed for security issues, such as support vector machine (SVM) [6], [7], k-nearest neighbor (kNN) [8], [9], decision tree (DT) [10], [11], k-means [12], [13], and others [14]–[16]. Most recently, deep neural network models have been utilized for intrusion detection in cloud, fog, and several IoT based structures, such as deep recurrent neural network (RNN) [17], multi-layered perceptron neural network [18], Restricted Boltzmann Machines (RBM) [19], convolutional neural networks (CNN) [20], and others [21]. In addition, DL models can be used in problem in IoT environment such as generative adversarial networks (GANs) [22] to protect user's private information and improve the device utility.

Feature selection techniques have showed significant performance in IDS with various classifiers. In recent years, metaheuristics (MH) optimization algorithms have been adopted for various complex problems, including feature selection. They have been also applied for intrusion detection, example, genetic algorithm [23]–[25], particle swarm optimization (PSO) [26], gery wolf optimizer (GWO) [27], [28], random harmony search (RHS) [29], and crow search algorithm (CSA) [30].

In this paper, we propose an efficient IDS system relying on the advancements of deep learning (DL) and MH optimization algorithms. First, we develop a simple yet effective feature extractor model with a convolutional neural network (CNN) as a backbone. The CNN model contains several convolution blocks to extract relevant and complex features. In addition, the CNN blocks are followed by a fully connected layer for feature extraction and intrusion detection (classification). Mainly, the CNN model is trained to classify each attack type with the aim of reaching the best accuracy and learn meaningfully representations from the raw data. Later, the raw data will be fed again to the trained model to convert raw samples attributes to the learned features by the CNN. Second, we develop a new feature selection (FS) method to enhance IDS classification using a new variant of the Transient search optimization (TSO). TSO is a new optimization MH algorithm proposed by Qais *et al.* [31]. It was inspired by the transient behavior of switched circuits with storage elements, for example, capacitance and inductance. As described in [31]. TSO was applied to solve various optimization problems, and it showed competitive performance. For example, it has been applied to estimate the parameters of photovoltaic as in [31], and it has been established its performance against other methods. In [32], a modified version of TSO has been presented and applied to find the solution for different global optimization problems. In [33], authors used TSO to determine the optimal allocation of multiple distributed generators in the radial electrical distribution network. With these advantages achieved by TSO in real-world applications, it still requires more improvements, especially the process of balancing between the exploration and exploitation during the searching process. Therefore, we adopt a new enhanced version of the TSO as an FS method to enhance IDS. We use a well-known technique to boost the search performance of the TSO and to avoid its limitations, such as trapping at local optima. The differential evolution (DE) [34] is used to balance between the exploitation and exploration phases and enhance the diversity of the agents.

The proposed model starts by training a CNN model as a feature extractor for each evaluated dataset. After extracting the feature vector with a fixed length for each sample, the relevant features are determined using the modified TSODE. The developed CNN model and TSODE are evaluated using three public datasets, KDDCup-99, NSL-KDD, and BoT-IoT. Additionally, TSODE was compared to the traditional TSO alongside several well-known optimization algorithms used for FS applications. The application of DE has significantly enhanced the performance of the traditional TSO, and the proposed TSODE showed significant performance in all tested datasets.

To sum up, in this paper, we present the following contributions:

- Propose an efficient IDS approach using advantages of deep learning and MH optimization algorithms.
- Develop a CNN-based feature extraction method to extract relevant features from the input datasets.
- Propose a new variant of the TSO algorithm, called TSODE, using the DE operators that are employed to enhance exploration phases and diversity of the agents to of the traditional TSO and avoiding its shortcomings.
- Evaluate the TSODE with extensive comparison to state-of-art methods using three public datasets.

The structure of the rest of this paper is given as: In section II, the related work is given. Section III introduces the basic steps of Transient search optimization and Differential Evolution. Section IV describes the developed IoT security model. Section V presents the results and discussion. Finally, the conclusion and future works are discussed in Section VI.

II. RELATED WORKS

In this section, we highlight a number of studies that applied metaheuristic algorithms for intrusion detection. Saljoughi *et al.* [35] proposed an attack intrusion detection scheme for cloud computing using a combined model of deep learning and swarm intelligence. They used a Multilayer Perceptron (MLP) Neural Networks and particle swarm optimization(PSO) algorithm for attacks and intrusion detection. Both KDD-CUP and NSL-KDD datasets were applied in the evaluation experiments, and the proposed scheme showed enhanced accuracy in attacks and intrusion detection.

In [36], the artificial bee colony (ABC) was employed to enhance the classifier to detect and tackle denial-of-service (DOS) attacks in the cloud. The prediction results were enhanced by applying the ABC algorithm compared to quantum-inspired particle swarm optimization technique (QPSO), with an average detection rate of 72.4%. Dash [37] proposed two methods to train artificial neural networks (ANN) for intrusion detection using metaheuristic optimization algorithms. The first one is using the gravitational search (GS) algorithm, where the second one is by combining both GS and PSO. GS and GS-PSO were employed to train the ANN, and they confirmed their high quality for the NSL-KDD dataset in comparison with several approaches used to train ANN, such as gradient descent, PSO, and genetic algorithm (GA).

In [38], a feature selection method based on GA for intrusion detection was proposed. The GA was applied with a fuzzy support vector machine (SVM) and showed significant performance when was evaluated with KDD Cup 99 datasets. Nazir and Khan [39] proposed a new feature selection using the Tabu Search (TS) algorithm to train the Random Forest (RF) classifier to build a robust intrusion detection system. The proposed system, called TS-RF, was evaluated using UNSW-NB15 dataset. The evaluation outcomes showed that the TS outperformed several feature selection methods, and the proposed TS-FS enhanced the classification accuracy.

SaiSindhuTheja and Shyam [30] proposed a new Detection of Denial of Service (DoS) attack detection system using a modified Crow Search Algorithm (CSA) for feature selection. The Opposition Based Learning (OBL) is combined with the CSA to boost its performance. Then, the Recurrent Neural Network (RNN) is applied as a classifier. The evaluation results confirmed the competitive performance of the proposed feature selection method using the improved CSA and high classification accuracy of the RNN with CSA. Mayuranathan *et al.* [29] proposed an improved intrusion detection system with a new feature selection technique using the Random Harmony Search (RHS) optimization algorithm. The Restricted Boltzmann Machines were applied as a classifier for detection Distributed Denial-of-Service (DDoS). The evaluation was implemented with KDD'99 datasets, and the proposed system obtained significant performance. RM *et al.* [28] developed an efficient feature selection approach using a hybrid of principal component analysis (PCA) and grey wolf optimization (GWO) algorithm. The proposed approach, called PCA-GWO, is employed to optimize the deep neural network to improve its performance in the application of intrusion detection on the internet of medical things applications. The classification and prediction accuracy verified the successful application of the PCA-GWO as a feature selection method that boosts the classification accuracy in comparison to other methods. Furthermore, Alsaedi *et al.* [40] proposed a new dataset, called TON_IoT. They used several classification methods to evaluate the collected dataets, and they found that RF and Classification and Regression Trees (CART) obtained the best results, whereas LSTM and KNN came in the second rank compared to other classification methods.

III. BACKGROUND

A. TRANSIENT SEARCH OPTIMIZATION

In this section, the steps of the transient search optimization (TSO) are introduced [31]. In general, TSO simulates the transient behavior of switched electrical circuits that contains capacitance and inductance. In addition, the exploration phase in TSO represents the oscillations of the second-order RLC circuits around zero. Meanwhile, the exploitation represents the exponential decaying of the first-order discharge.

The first step in the mathematical modeling of TSO is to generate the population (X) of N agents and determine the best of them X_b. The next step is to update these agents using the operators of TSO as defined in the following equation.

$$X_i = \begin{cases} X_b + (X_i - C_1 \times X_b) \times e^{-T} & if\ r_1 < 0.5 \\ X_b + e^{-T}\,(cos(2\pi T) + sin(2\pi T)) \times D_{ib} & Otherwise \end{cases} \tag{1}$$

In Eq. (1), $D_{ib} = |X_i - C_1 \times X_b|$ C_1 and T are random coefficients that defined as in Eq. (2) and Eq. (3), respectively.

$$T = 2 \times Z \times r_2 - Z \tag{2}$$

$$C_1 = k \times Z \times r_3 + 1 \tag{3}$$

where r_2 and r_3 are random numbers generated from [0, 1]. While, Z represents a random number changed from 2 to 0 as follows:

$$Z = 2 - 2 \times (\frac{t}{t_{max}}) \tag{4}$$

where t and t_{max} refer to the current generation and the total number of generations respectively. The next step is to compute the fitness value of each solution and update the best agent X_b. Then the terminal conditions are tested, and when they are satisfied, then the process of updating the agents is stopped and return the best agent.

B. DIFFERENTIAL EVOLUTION

DE is a well-known optimization algorithm [34], which was adopted to solve various problems due to its advantages, such as the fast convergence and the fast implementation (it requires less computing time). The DE works by randomly initializing X population; after that, it updates the population by applying *crossover* and *mutation* operators. *Mutation* operator is employed to the current X_i as in Eq. (5):

$$z_i^t = X_{r_1}^t + F \times (X_{r_2}^t - X_{r_3}^t), \tag{5}$$

In which r_1, r_2, and r_3 are the random indices, and they are different from the current index i. Where $F > 0$ is the mutation scaling factor, and the iteration number is represented by t.

Algorithm 1 Steps of TSO

```
Input: the number of agents N.
generate initial population X.
repeat
    Compute fitness value of each agent X_i, i = 1, 2, ..., N
    Determine the best of them (i.e., X_b).
    for i = 1 : N do
        Update the value of T and C.
        Update X_i using Eq. (1).
    end for
until Stop conditions met
Return X_b.
```

Thereafter, *crossover* operator is employed to produce a new vector v_i as:

$$v_i^t = \begin{cases} z_i^t & if\ r \leq C_r \\ X_i^t & otherwise \end{cases} \tag{6}$$

in which r represents a random value at [0, 1], where C_r represents the *crossover* probability.

Furthermore, the current individual can be replaced by the generated individual v_i^t, if the v_i^t obtained a better fitness, as follows:

$$X_i^{t+1} = \begin{cases} v_i^t & if\ f(v_i^t) < f(X_i^t) \\ X_i^t & otherwise \end{cases} \tag{7}$$

The steps are repeated till meeting stop criterion.

IV. PROPOSED MODEL

The framework of the developed Internet of Things network security is given in Figure 3. The proposed framework is composed of two phase which are the feature extraction phase using CNN model and the selection of most important features using the developed TSODE algorithm. The developed TSODE, depends on improving the behavior of traditional TSO using the operators of DE, as shown in Figure 1. DE is applied to enhance the process of balancing between exploration and exploitation during the search for the feasible region and best solution. This will be reflected in the quality of the final solution that will use an optimal subset of features which lead to increasing the prediction of intrusion detection in the IoT environment. In the following sections a detailed explanation of the framework will be given.

A. REPRESENTATION OF COLLECT IoT DATASET

In this section, the basic representation of traffic data of the IoT that will be used as input to the next stage of the developed method is given. Consider TS represents the sample of the traffic of IoT, and it formulated as:

$$TS = \begin{bmatrix} tf_{11} & tf_{12} & \dots & tf_{1d} \\ tf_{21} & tf_{22} & \dots & tf_{2d} \\ \dots & \dots & \dots & \dots \\ tf_{n1} & tf_{n2} & \dots & tf_{nd} \end{bmatrix} \tag{8}$$

where TS_i is the ith set of features (i.e., $[tf_{11}, tf_{12}m \dots, tf_{1d}]$) of traffic. n and d refer to the number of samples and features respectively. The next step is to normalize the dataset using the min−max normalization method that defined as:

$$DN_{ij} = \frac{tf_{ij} - min(TS_j)}{max(TS_j) - min(TS_j)} \tag{9}$$

In Eq. (9), tf_{ij} denotes the value of jth feature of the ith sample. In this study, we used the min-max normalization since it transforms the given data with varying scales. Therefore, no specific feature can dominate the statistics. In addition, it doesn't make hypothesis about the distribution of the datasets for example, artificial neural networks and k-nearest neighbours. In contrast, the Standardization is useful in the case the data following Gaussian distribution.

The normalized version of TS is given as:

$$NTS = \begin{bmatrix} DN_{11} & DN_{12} & \dots & DN_{1d} \\ DN_{21} & DN_{22} & \dots & DN_{2d} \\ \dots & \dots & \dots & \dots \\ DN_{n1} & DN_{n2} & \dots & DN_{nd} \end{bmatrix} \tag{10}$$

These normalized samples are used as input to the DL model to extract the features from it. This DL is discussed in the following stage.

B. CONVOLUTIONAL NEURAL NETWORK FOR FEATURE EXTRACTION

Nowadays, structured and unstructured data forms (audio, image, and text) are modeled using DL techniques which employes supervised (discriminative learning) and unsupervised strategies (generative learning) [41]–[43]. These strategies can help the DL model to learn and extract complex representations and features. The type of layers can differentiate DL models and the learning mechanism [44], [45]. This section introduces an overview of a simple yet effective convolutional neural network (CNN) model used in the experiments to extract relevant features from the exploited data.

The CNN models have flexible architectures and are well-known feature extractors used in various applications and tasks. The main characteristic of these models is the shared weight strategy among multiple computation layers [46]. A basic CNN architecture can contain convolutional, activation, pooling, and fully connected layers stacked in a specific topology. Based on the depth of the topology, the model, the extracted features can range from low-level features to more complex features.

As shown in Fig. 2, the proposed CNN architecture that acts as a feature extractor contains two convolution blocks separated with a pooling layer. In addition, the model uses four fully connected layers for feature extraction and prediction task. The input data (NTS) is fed to the CNN block with a 1D convolution operation that produces activation maps when applying a fixed kernel of 1×3.

A Rectified Linear Unit (ReLU) [47] is used as the convolution activation function to learn more complex patterns

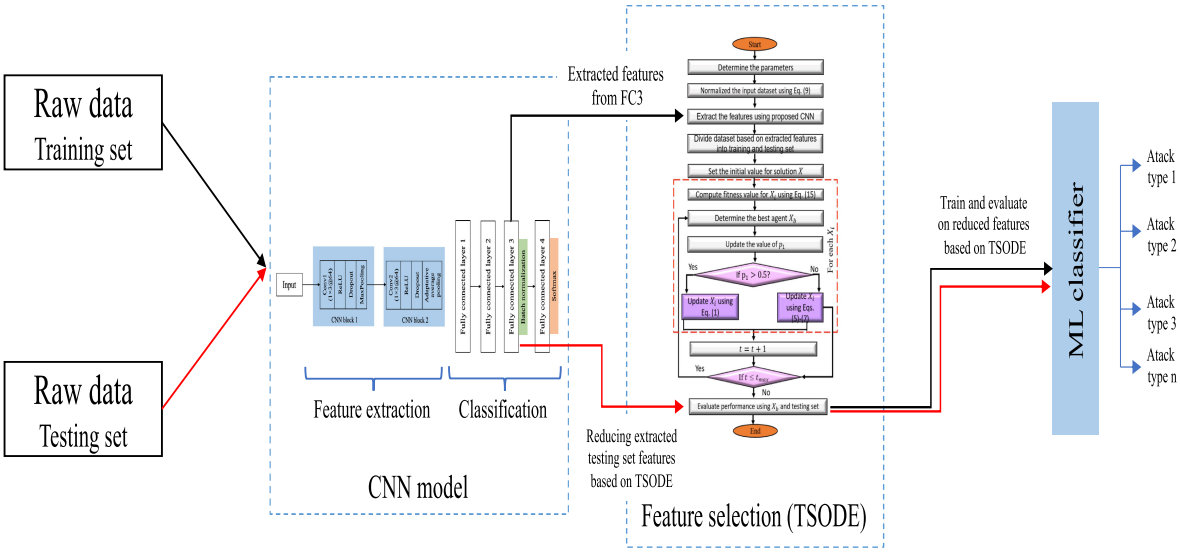


FIGURE 1. Steps of developed model for IoT security.

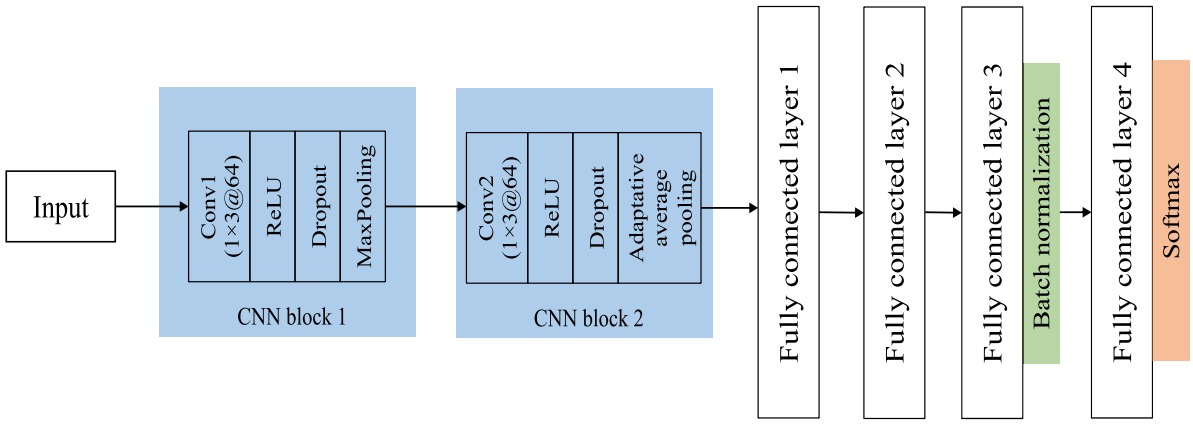


FIGURE 2. Proposed CNN architecture for feature extraction.

which is defined as in in Eq. (11).

$$z_j^l = ReLU(Z_j^l) \tag{11}$$

where z_j^l represents the output activation map of the l layer and j channel. Z_j^l can be obtained as in Eq. (12).

$$Z_j^l = \sum_{i \in M_j} z_j^{l-1} k_{ij}^l + b_j^l \tag{12}$$

where y_j^{l-1} is the previous output activation map of precedent layer. The convolution kernel weights and the bias value are defined as k_{ij}^l and b_j^l, respectively.

The ReLU activation function in the CNN block is followed by dropout to overcome the overfitting problem. The output of the CNN blocks is downsampled using pooling layers of types max-pooling and adaptive average pooling layer [48]. The downsampling operation helps the model train faster by minimizing the number of training parameters and paying attention to the most relevant features.

Three fully connected layer $(FC1 - 128) \rightarrow (FC2 - 128) \rightarrow (FC3 - 64)$ are used for feature extraction followed by batch normalization (BN). FC3 layer is used to extract the features to be used in the feature selection phase. The extracted features are the automatically learned features by the CNN (on raw data/samples) at the third fully connect layer after training the network on the dataset with the aim of maximizing the attack classification accuracy. It is like converting the raw data/samples representations to more meaningful and manageable representations with lower dimension. A softmax layer for prediction follows the last fully-connected layer (FC4) of size 64. BN is used to speed up the model's convergence, whereas the softmax activation function is used in the output layer to predict the class (attack type) normalized probability for each sample.

C. FEATURE SELECTION

In this stage of the developed FS model as shown in Figure 1, the relevant features are selected according to their quality to improve intrusion detection in IoT. This is achieved by using a modified version of TSO based on the operator of DE that used a local optimizer technique. The developed FS method, named TSODE, starts by constructing the initial population X, which contains N agents. Then converting each agent into its binary form and reduce the training set by removing the features that correspond to zeros inside this binary form. The next process is to compute the performance of the selected feature using the error classification of the KNN classifier. After that, the best agent that has the best fitness value is allocated. According to this best agent and the operators of TSO and DE, the agents inside the current population are updated until they reached the optimal solution.

1) CONSTRUCTING INITIAL POPULATION

The developed TSODE starts by dividing the dataset into 80% and 20% from the total samples of datasets which represents training and testing sets, respectively. Then TSODE uses Eq. (13) to form the initial value for a set of N agents X, which represents the initial population.

$$X_i = LB + rand(1, D) \times (UB - LB) \tag{13}$$

where D is the number of extracted features, and it represents the dimension of each agent. $rand(1, D)$ denotes random D values generated from [0, 1]. LB and UB refer to the limits of the search domain.

2) UPDATING POPULATION

This stage begins by converting each agent X_i into its Boolean form BX_i using Eq. (14).

$$BX_{ij} = \begin{cases} 1 & if\ X_{ij} > 0.5 \\ 0 & otherwise \end{cases} \tag{14}$$

According to Eq. (14), the number of features inside the training set is decreased by removing those features corresponding to zeros; then computing the fitness value for each agent X_i using the following equation.

$$Fit_i = \lambda \times \gamma_i + (1 - \lambda) \times (\frac{|BX_i|}{D}) \tag{15}$$

In Eq. (15), γ_i denotes the error of classification that calculated using KNN classifier based on training set. $\lambda \in [0, 1]$ is random weights used to balance between the error of classification and the ratio of relevant features ($\frac{|BX_i|}{D}$).

For clarity Eqs. (14)-(15), consider the current agent X_i has seven features (dimension) is representing as $U1_i$ = [0.0975, 0.2785, 0.5469, 0.9575, 0.9649, 0.1576, 0.9706]. By applying Eq. (14) then BX_i = [0, 0, 1, 1, 1, 0, 1]. This means that the third, fourth, fifth, and seven features are chosen as relevant features and used to reduce the training set and using Eq. (15) to assess this selecting features.

The next step is to find the best agent X_b which has the best fitness value Fit_b, then using X_b to update the current agents with the operators of TSO and DE. This process is performed by giving these operators the ability to work in a competitive manner, which leads to maintaining the agents' diversity. The updating process is conducted using the following equation.

$$X_i = \begin{cases} Use\ Eq.\ (1) & if\ p_1 < 0.5 \\ Use\ \ Eqs.\ (7) - (5) & otherwise \end{cases} \tag{16}$$

where p_1 is random probability used to balance between the operators of TSO and DE.

3) TERMINAL CONDITIONS FOR LEARNING STAGE

In this stage, the stopping conditions are checked, and in case they are not met. Then the updating stage is performed again. Otherwise, the X_b is returned, which is used in the next stage to reduce the testing set.

4) EVALUATION USING TESTING SET

To assess the ability of the developed TSODE as an FS method, the best agent X_b is used to remove the irrelevant features from the testing set, then computing the quality of the classification process using different performance metrics based on the reduced features. The full steps of the developed IoT model to detect the intrusion are given in Algorithm 2.

Algorithm 2 Developed Feature Selection for Security of IoT

```
Input: t_max: maximum number of iterations, and N: number of solutions.
Normalized the input dataset using Eq. (9).
Extract the features using proposed CNN model as in described in section IV-B.
Divide the dataset based on extracted features into training and testing set.
Construct initial population X using Eq. (13).
Set t = 1.
while t <= t_max do
    Use Eq. (14) to obtain Boolean form for each agent X_i.
    Apply Eq. (15) to compute the fitness value Fit_i for each X_i.
    Allocate the best agent X_b.
    Update the value of p_1.
    if p_1 < 0.5 then
        Update X_i using Eq. (1).
    else
        Update X_i using Eqs. (5)-(7).
    end if
    t = t + 1.
end while
Reduce the testing set using the relevant feature (corresponding to ones) inside X_b.
Output: Return by X_b and the value of performance metrics.
```

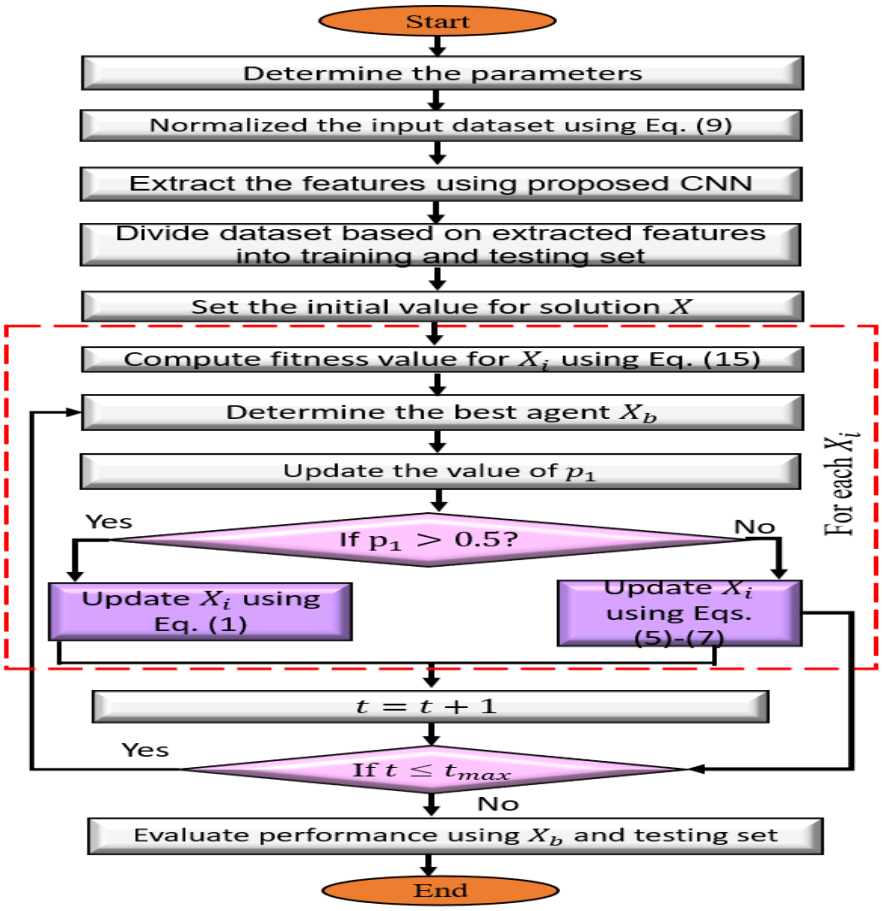


FIGURE 3. The proposed TSODE as FS method.

TABLE 1. Confusion matrix.

	Predected class	
Actual class	**Positive**	**Negative**
Postive	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP	True Negative (TN)

V. EXPERIMENT RESULTS AND DISCUSSION

A. PERFORMANCE MEASURES

To assess the ability of the developed method to detect the intrusion in IoT environment, a set of performance measures is used. For example, accuracy, sensitivity, specificity, and F-measure, and each of them depends on the confusion matrix defined in Table 1. The definition of each measure is formulated as:

- *Average Accuracy* (AV_{Acc})*:* The accuracy metric represents the rate of correct detection of the intrusion, and it is formulated as:

$$AV_{Acc} = \frac{1}{N_r}\sum_{k=1}^{N_r} Acc_{Best}^{k}, \quad Acc_{Best} = \frac{TP + TN}{TP + FN + FP + TN} \tag{17}$$

where $N_r = 30$ denotes the number of runs.

- *Average Recall* (AV_{Sens})*:* It is also named as true positive rate (TPR), which denotes the percentage of predicting positive intrusion, and it is given as:

$$AV_{Sens} = \frac{1}{N_r}\sum_{k=1}^{N_r} Sens_{Best}^{k}, \quad Sens_{Best} = \frac{TP}{TP + FN} \tag{18}$$

- *Average Precision* (AV_{Prec})*:* This shows the percentage of truly positive out of all the positive predicted samples, which is given as:

$$AV_{Prec} = \frac{1}{N_r}\sum_{k=1}^{N_r} Prec_{Best}^{k}, \quad Prec_{Best} = \frac{TP}{FP + TP} \tag{19}$$

- *Performance Improvement Rate (PIR)*: It is used to measure the improvement rate achieved by the developed method and it is defined as:

$$PIR = \frac{M_{TSODE} - M_{Alg}}{M_{TSODE}} \times 100 \tag{20}$$

where M_{TSODE} represent the performance measure value (i.e., Accuracy, Recall, Precision, and F1-measure)

TABLE 2. Selected CNN architecture hyper-parameters.

Block	Hyper-parameters	Value
CNN block 1	Kernel size	3
	Filters	64
	Stride	1
	ReLU	-
	Dropout	0.5
Max pooling	Pooling size	2
CNN block 2	Kernel size	3
	Filters	64
	Stride	1
	ReLU	-
	Dropout	0.5
Adaptive average pooling	Pooling size	2
Fully connected (FC1)	Number of neurons	128
Fully connected (FC2)	Number of neurons	128
Fully connected (FC3)	Number of neurons	64
Batch normalization	Number of features	64
Fully connected (FC4)	Number of neurons	64

of the developed TSODE and compared algorithm, respectively.

B. EXPERIMENTAL SETUP

In our experiments, the CNN setup used Adam [49] with a learning rate equal to 0.005 is selected as the network optimizer over a batch size equal to 2024 and 100 epochs. Table 2 lists the network topology and parameters. In addition, the performance of developed TSODE as feature selection is compared with other MH techniques including particle swarm optimization (PSO) [50], multiverse optimization algorithm (MVO) [51], Grey wolf optimizer (GWO) [52], moth flame optimization (MFO) [53], whale optimization algorithm (WOA) [54], Firefly algorithm (FFA) [55], Bat algorithm [56], and traditional TSO. The parameters of these MH methods are set according to the original implementation.

C. DATASET DESCRIPTION

Three datasets, namely KDDCup-99, NSL-KDD, and BoT-IoT were used to evaluate the proposed model in binary classifications. Most researchers regularly utilize these datasets to benchmark the performance of their network intrusion models. The challenge was to classify the connection records as either attack (intrusion) or benign.

1) *KDDCup-99:* The dataset was collected from the 1998 DARPA intrusion detection challenge dataset created by the MIT Lincon laboratory. A set of 1000's UNIX machines and 100's users were used for ten weeks to capture the network traffic data. The captured data was stored in tcpdump format to create the KDDCup 1998 dataset. A feature extraction operation has been conducted on the processed tcpdump data using the mining audit data for automated models for the ID (MADMAID) framework. We used 10% of the full KDDCup-99 dataset in our experiments, and the connection records were normalized. As shown in Table 3, KDDCup-99 contains of 5 attack types and 41 features. The features are grouped into three main categories, including basic features, which contain the packet capture (Pcap) files, content features that contain the full payload of TCP/IP packets information, and time-based traffic features with 2 seconds overlapping window.
2) *NSL-KDD:* A refined version from KDDCup-99 after removing redundant connection records. In addition, The CSV format of the dataset with 41 features and five attack types. Table 3 report the detailed statistics of the dataset.
3) *BoT-IoT:* Industrial IoT (IIoT) smart home appliances were used to collect IIoT traffic samples to create the Bot-IoT dataset [57] in the Cyber

TABLE 3. Description of KDDCup-99 and NSL-KDD datasets (10%).

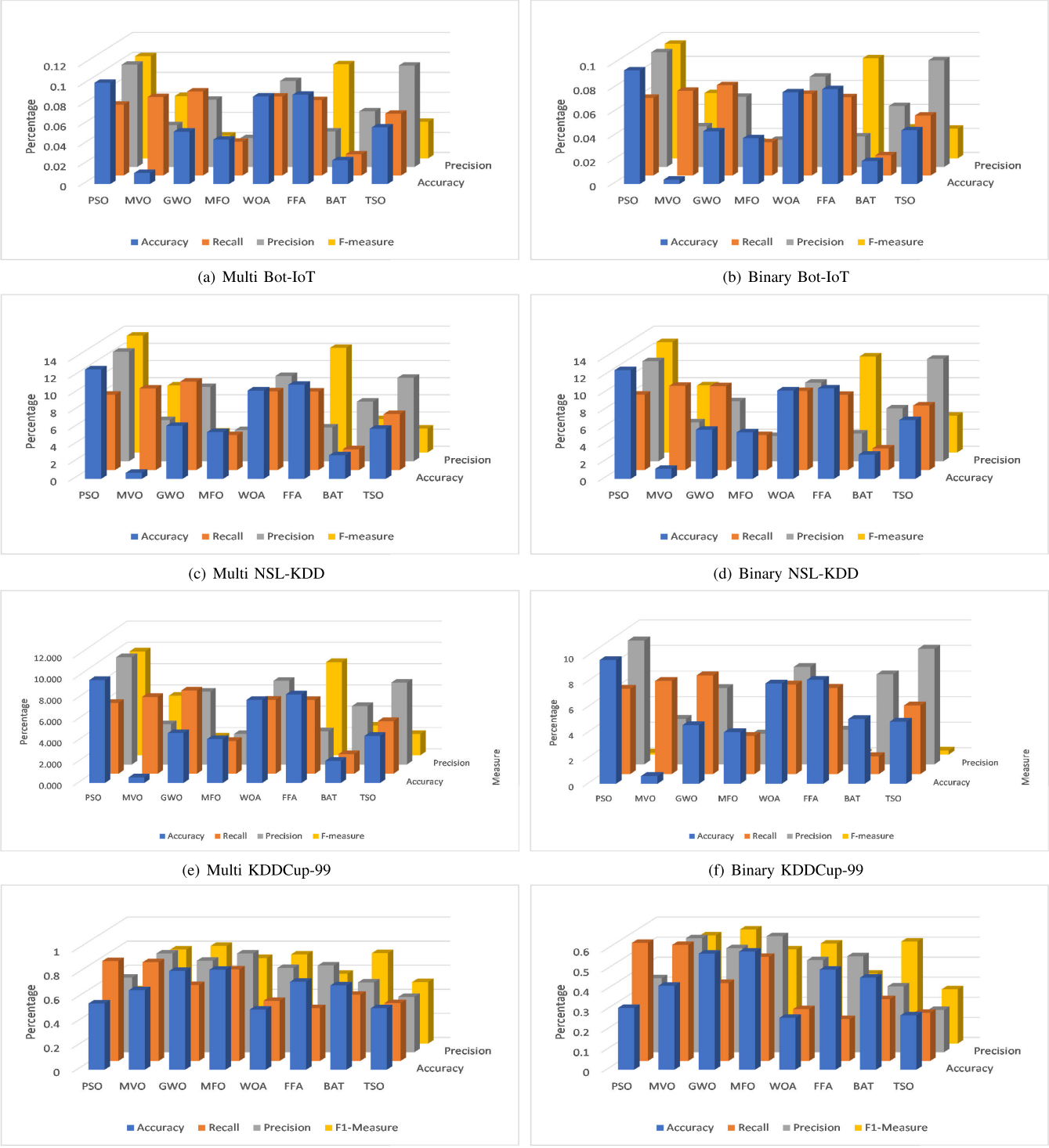
Attack Type	10% Connection Records			
	KDDCup-99		NSL-KDD	
	Train	Test	Train	Test
Normal	97,278	60,593	67,343	9,710
DoS	391,458	229,853	45,927	7,458
Probe	4,107	4,166	11,656	2,422
R2L	1,126	16,189	995	2,887
U2R	52	228	52	67
Total	**494,021**	**311,029**	**125,973**	**22,544**

TABLE 4. Description of Bot-IoT dataset (5%).

5% IIoT Traffic Records		
Attack Type	Train	Test
DDoS	1,541,315	385,309
DoS	1,320,148	330,112
Reconnaisance	72,919	18,163
Normal	370	107
Theft	65	14
Total	**2,934,817**	**733,705**

TABLE 5. Description of CICIDS-2017 dataset.

CICIDS-2017 Combined Four CSV Records		
Attack Type	Train	Test
Benign	727,397	163,572
DDoS	112,901	25,388
FTP-Patator	6997	1574
PortScan	140,043	31,492
SSH-Patator	5201	1169
Web Attack Brute Force	1329	299
Web Attack Sql Injection	19	4
Web Attack XSS	575	129
Total	**904,056**	**223,627**



(a) Multi Bot-IoT (b) Binary Bot-IoT

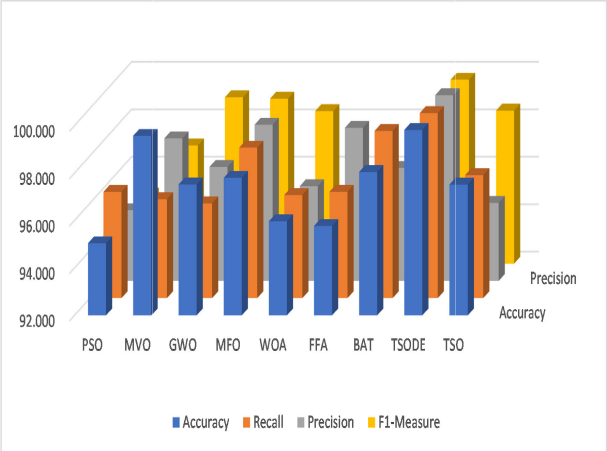
(c) Multi NSL-KDD (d) Binary NSL-KDD

(e) Multi KDDCup-99 (f) Binary KDDCup-99

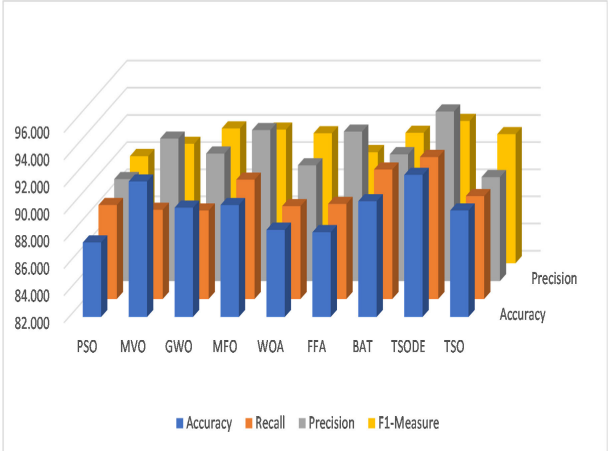
(g) Multi CICIDS-2017 (h) Binary CICIDS-2017

FIGURE 4. **Percentage difference between developed method and other MH techniques over the tested datasets in binary and multi classification.**

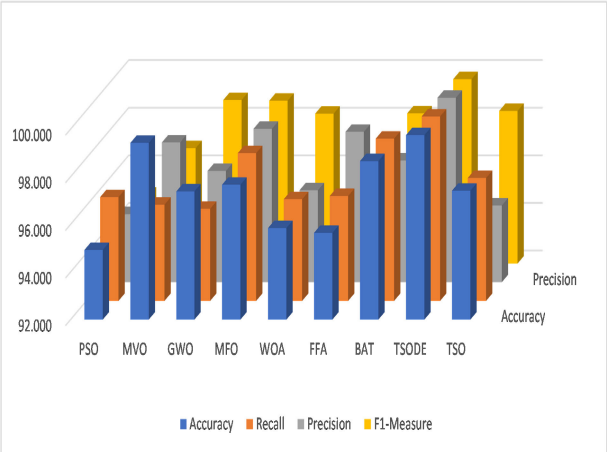
Range Lab of The center of UNSW Canberra Cyber. Smart IIoT devices including thermostats, motion-controlled lights, remotely controlled garage, fridges and freezers, and weather monitoring systems. The data is presented in two versions, including the full version, which contains over 72 million records, and the 10%, which consists of approximately 3.6 million records. We decide to experiment with the proposed model on the 5% of the entire dataset with a group of best ten features. Table 4 lists the train and test sets



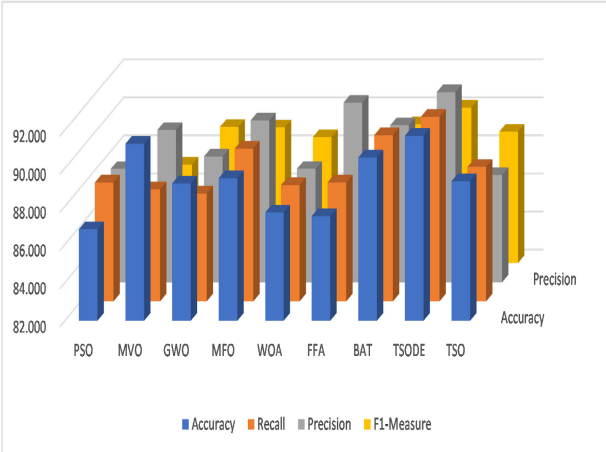
(a) Average of train set for binary classification



(b) Average of test set for binary classification



(c) Average of train set for Multi-classification



(d) Average of test set for Multi-classification

FIGURE 5. Average of performance measures overall the four datasets.

records of the IIoT traffic categorized into five main classes.

4) *CICIDS-2017:* The intrusion detection dataset CICIDS-2017 [58] was collected at the Canadian Institute for Cybersecurity (CIC), University of New Brunswick, Canada. The CICIDS-2017 consists of many records, which are over 1.5 million, which simulate true real-world data (PCAPs). The dataset covers various attack types, including scan attacks, brute force, DoS, DDoS, infiltration, heart-bleed, bot, and Web-based. The dataset PCAP traffic files have been used to build the CSV files using the resulted network traffic analysis by CICFlowMeter. The CICFlowMeter software analyzes different connection protocols (HTTP, FTP, SSH, and email protocols) of 25 user behaviors. The overall dataset saved in CSV files contains 80 network traffic features and flows labels. To train and evaluate our proposed framework, we have selected four CSV files to build the used dataset in our experiment: Tuesday-WorkingHours, Thursday-WorkingHours-Morning-WebAttacks Friday-WorkingHours-Afternoon-PortScan, and Friday-WorkingHours-Afternoon-DDos. The combined files resulted in a total of 1,127,683 records distributed over benign and seven attack types. Table 5 lists the train and test sets records and attack types used in our experiments.

D. RESULTS AND DISCUSSION

In this section, the comparison results between the developed TSODE and the other MH techniques are discussed. Tables 6-8 show the average of different metrics for the tested dataset used in our comparison (i.e., BoT-IoT, KDDCup-99, and NSL-KDD). It can be observed in the case of Multi-classification of BoT-IoT as given in Table 6 that most of the MH techniques nearly have the same performance during the training stage. However, PSO provides high performance measures. In addition, the developed

TABLE 6. Comparison results of developed method using Bot-IoT dataset.

Multi classification								
	Train				Test			
	Accuracy	Recall	Precision	F1-Measure	Accuracy	Recall	Precision	F1-Measure
PSO	**99.483**	**99.483**	**99.483**	**99.483**	98.942	98.972	98.941	98.940
MVO	99.468	99.468	99.468	99.468	99.031	98.964	99.000	98.980
GWO	99.477	99.477	99.476	99.476	98.990	98.959	98.975	99.019
MFO	99.480	99.480	99.480	99.480	98.998	99.009	99.013	99.020
WOA	99.472	99.472	99.472	99.472	98.956	98.964	98.957	99.005
FFA	99.479	99.479	99.478	99.478	98.954	98.968	99.007	98.949
BAT	99.475	99.475	99.475	99.474	99.019	99.021	98.987	99.012
TSODE	99.474	99.474	99.473	99.473	**99.042**	**99.042**	**99.042**	**99.042**
TSO	99.460	99.460	99.459	99.459	98.986	98.981	98.941	99.005
Binary classification								
	Train				Test			
	Accuracy	Recall	Precision	F1-Measure	Accuracy	Recall	Precision	F1-Measure
PSO	99.899	99.929	99.898	99.898	99.898	99.928	99.896	99.896
MVO	99.990	99.923	99.959	99.939	99.989	99.922	99.958	99.937
GWO	99.950	99.919	99.935	99.979	99.948	99.917	99.933	99.977
MFO	99.956	99.966	99.971	99.978	99.954	99.964	99.969	99.976
WOA	99.918	99.926	99.919	99.967	99.916	99.924	99.916	99.965
FFA	99.915	99.928	99.968	99.910	99.913	99.927	99.966	99.908
BAT	99.975	99.977	99.943	99.968	99.973	99.975	99.941	99.966
TSODE	**99.994**	**99.994**	**99.994**	**99.993**	**99.992**	**99.992**	**99.992**	**99.991**
TSO	99.949	99.944	99.905	99.969	99.947	99.942	99.903	99.967

TABLE 7. Comparison results of developed method using KDDCup-99 dataset.

Multi-classification								
	Train				Test			
	Accuracy	Recall	Precision	F1-Measure	Accuracy	Recall	Precision	F1-Measure
PSO	90.447	93.458	90.358	90.358	82.783	85.793	84.640	83.109
MVO	99.515	92.835	96.483	94.433	91.615	84.935	86.649	84.480
GWO	95.513	92.383	94.062	98.482	87.618	84.488	84.131	88.533
MFO	96.073	97.123	97.631	98.371	88.175	89.225	87.763	88.420
WOA	92.275	93.126	92.414	97.304	84.375	85.225	82.501	87.351
FFA	91.988	93.368	97.328	91.538	84.318	85.698	**91.609**	84.285
BAT	98.007	98.247	94.847	97.337	90.347	90.587	89.134	**90.093**
TSODE	**99.951**	**99.951**	**99.950**	**99.949**	**92.064**	**92.064**	89.943	90.007
TSO	95.439	94.919	91.027	97.437	87.536	87.016	80.791	87.479
Binary Classification								
	Train				Test			
	Accuracy	Recall	Precision	F1-Measure	Accuracy	Recall	Precision	F1-Measure
PSO	90.449	93.459	90.359	90.359	82.775	85.785	84.638	92.702
MVO	99.519	92.839	96.489	94.439	91.844	85.164	90.765	92.701
GWO	95.518	92.388	94.068	98.488	87.860	84.730	88.357	92.716
MFO	96.079	97.129	97.639	98.379	88.413	89.463	91.922	92.710
WOA	92.278	93.128	92.418	97.308	84.608	85.458	86.699	92.705
FFA	91.987	93.367	97.327	91.537	84.327	85.707	91.614	92.713
BAT	94.992	98.662	92.922	91.782	87.384	91.055	87.280	92.751
TSODE	**99.930**	**99.930**	**99.930**	**99.930**	**92.451**	**92.451**	**94.320**	**92.851**
TSO	95.298	94.592	90.825	97.332	87.593	87.090	85.280	92.541

TSODE has the best accuracy, specificity, and sensitivity, and F1-measure. In the binary case of Bot-IoT dataset, it can be noticed that the developed TSODE has provided better results in training and testing set in terms of all performance measures. Whereas, Figure 4(a)-4(b) depict the performance improvement rate (*PIR*) of the developed

TABLE 8. Comparison results of developed method using NSL-KDD.

	Multi classification							
	Train				Test			
	Accuracy	Recall	Precision	F1-Measure	Accuracy	Recall	Precision	F1-Measure
PSO	90.118	93.128	90.020	90.019	66.092	69.102	68.913	61.940
MVO	99.182	92.502	96.145	94.093	75.224	68.544	75.200	66.098
GWO	95.182	92.052	93.724	98.143	71.066	67.936	72.151	69.948
MFO	95.745	96.795	97.297	98.035	71.626	72.676	76.122	69.844
WOA	91.947	92.797	92.080	96.968	67.951	68.801	71.131	68.907
FFA	91.660	93.040	96.991	91.201	67.437	68.817	75.873	62.944
BAT	97.669	97.909	94.501	96.989	73.671	73.911	73.501	68.905
TSODE	**99.612**	**99.612**	**99.603**	**99.602**	**75.751**	**75.751**	**78.988**	**71.692**
TSO	95.078	94.558	90.657	97.067	71.330	70.810	71.298	69.697
	Binary classification							
	Train				Test			
	Accuracy	Recall	Precision	F1-Measure	Accuracy	Recall	Precision	F1-Measure
PSO	90.133	93.143	90.043	90.043	67.575	70.585	73.882	67.163
MVO	99.197	92.517	96.167	94.117	76.466	69.786	79.835	71.059
GWO	95.202	92.072	93.753	98.172	72.944	69.814	77.801	75.609
MFO	95.760	96.810	97.320	98.060	73.187	74.237	81.176	75.162
WOA	91.959	92.809	92.099	96.989	69.409	70.259	75.972	74.115
FFA	91.673	93.053	97.013	91.223	69.218	70.598	80.944	68.451
BAT	97.693	97.933	94.533	97.023	75.192	75.432	78.473	74.197
TSODE	**99.629**	**99.629**	**99.629**	**99.629**	**77.381**	**77.381**	**83.637**	**77.080**
TSO	95.091	94.571	90.681	97.091	72.078	71.558	73.656	73.786

TSODE method and other MH techniques. It can be seen, PIR in multi-classification variants from 0.011 to 0.101 in terms of accuracy and 0.0212 to 0.084 in terms of Recall. Whereas, 0.028 to 0.1021 and 0.0214 to 0.1021, respectively for Precision and F-measure. Also, for the binary classification the 0.0037 to 0.0946, 0.0168 to 0.0753, 0.0226 to 0.0955, and 0.0142 to 0.0955, respectively, in terms of Accuracy, Recall, Precision, and F-measure.

The comparison results between the developed method and other MH method using NSL-KDD dataset are given in Table 8 and Figure 4(c)-4(d). It can be seen from these results the superiority of developed TSODE over the other MH techniques when applied to either multi and binary classification of intrusion using NSL-KDD. The behavior of the developed TSODE in the learning stage is better than other models, as can be concluded from performance measures; the same can be noticed from the results of the testing set. In addition, the developed TSODE provides accuracy better than MVO with a difference nearly 0.528% and with a difference between it and PSO nearly 9%. In terms of Recall, Precision and F-measure, the developed TSODE is better than other models with difference variants from 1.840%, 2.866%, and 1.744% to 7.816%, 10.075%, and 9.752%, respectively. This can be noticed from Figure 4(c)-4(d).

Table 8 and Figure 4(g)-4(h) depict the comparison results between the developed TSODE and other MH models for KDDCup-99. From these results, one can be seen that the developed TSODE has better results in terms of all metrics when the training set of KDDCup-99 in case of multi-classification. However, when the testing set is applied, the BAT and FFA provide results better than other models in terms of F1-measure and Precision, respectively. While TSODE still provides better results in terms of accuracy, with a 0.4 difference between it and MVO. Moreover, in the case of binary KDDCup-99, the superiority of TSODE is observed from the comparison results in terms of all performance metrics over training and testing sets. Figure 5 shows the average of results in terms of metrics for overall the test datasets for each algorithm. It can be shown the high ability of the developed model to improve the detection of intrusion in both cases (i.e., multi and binary) of classification.

The comparative results of the developed method and other methods when applied to the CICIDS-2017 datasets are given In Table 9. It can be noticed that in both cases binary and multi-classification the developed method has better performance (nearly in all metrics) than other model, especially in the testing set. However, in general, the behaviour of most competitive FS methods is nearly the same in case of training set. In addition, the

Figure 6 shows that many of the misclassifications are due to the low frequent training samples provided to the CNN like U2R and R2L. Contrary, classes having a large number of training samples were well classified. In some cases, attacks such as PROBE were classified as Normal. This may be attributed to the difficulty of extracting characterized features by the CNN network for this attack type. Same observations can be noticed in Figure 7 where the CNN

TABLE 9. Comparison results of developed method using CICIDS-2017 dataset.

Multi classification								
	Train				Test			
	Accuracy	Recall	Precision	F1-Measure	Accuracy	Recall	Precision	F1-Measure
PSO	99.650	99.370	99.590	99.750	99.380	99.100	99.320	99.480
MVO	99.530	99.370	99.390	99.410	99.270	99.110	99.120	99.150
GWO	99.370	99.560	99.430	99.380	99.110	99.300	99.180	99.120
MFO	99.360	99.430	99.370	99.480	99.100	99.170	99.120	99.220
WOA	99.690	99.690	99.490	99.450	99.430	99.430	99.240	99.190
FFA	99.450	99.740	99.480	99.600	99.200	99.490	99.220	99.350
BAT	99.490	99.640	99.630	99.440	99.230	99.380	99.360	99.180
TSODE	**99.960**	**99.960**	**99.960**	**99.960**	**99.930**	**99.930**	**99.940**	**99.930**
TSO	99.680	99.710	99.750	99.680	99.420	99.450	99.480	99.420
Binary classification								
	Train				Test			
	Accuracy	Recall	Precision	F1-Measure	Accuracy	Recall	Precision	F1-Measure
PSO	99.687	99.407	99.627	99.387	99.687	99.407	99.627	99.787
MVO	99.577	99.417	99.427	99.457	99.577	99.417	99.427	99.457
GWO	99.417	99.607	99.477	99.427	99.417	99.607	99.477	99.427
MFO	99.407	99.477	99.417	99.427	99.407	99.477	99.417	99.527
WOA	99.730	99.531	99.537	99.470	99.737	99.737	99.537	99.497
FFA	99.497	99.601	99.517	99.470	99.497	99.787	99.517	99.647
BAT	99.537	99.647	99.667	99.472	99.537	99.687	99.667	99.487
TSODE	**99.735**	**99.665**	**99.765**	**99.485**	**99.996**	**99.996**	**99.996**	**99.996**
TSO	99.724	99.654	99.744	99.436	99.725	99.755	99.785	99.725

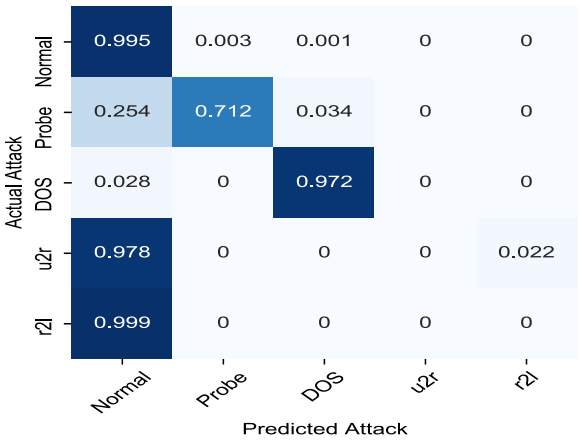


FIGURE 6. KDDCup99 dataset confusion matrix.

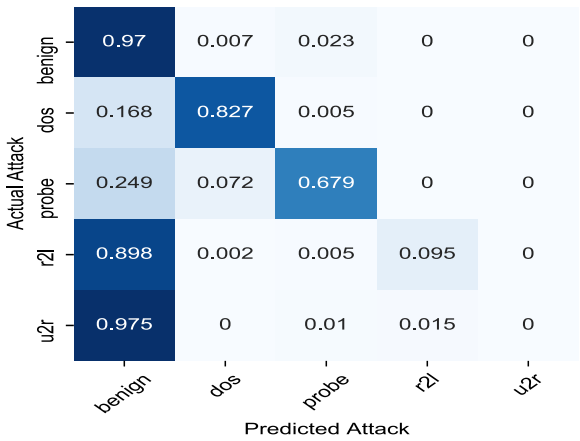


FIGURE 7. NSL-KDD dataset confusion matrix.

model was confused between three classes DOS, PROBE, and Normal, thus learning similar features as Normal class. The CNN models show better performance on the KDD-Cup99 dataset than the distilled NSL-KDD dataset, which helps to reduce the number of misclassifications in DOS. Likewise, the confusion matrix shown in Figure 9 for Bot-IoT dataset shows that Normal and Theft connection records were completely misclassified. This is due to the small size of the training connection records fed to the CNN network. The CNN model shows excellent ability to correctly classify attack types with significant training records such as DDoS, DoS, and Reconnaissance.

In addition, the performance of the developed method and other methods in terms of false positive rate (FPR) is given in Table 10. From these results it can be seen that the values of FPR for the TSODE are better than other algorithms in the binary and multi-class classification cases among all the tested four datasets (i.e., KDDCup-99, NSL-KDD, BoT-IOT, and CICIDS-2017). This indicate that the selected features using the proposed TSODE improve the detection performance of the classifier on each class comparing to other methods.

Table 11 shows the average of CPU time(s) for each algorithm among the tested set of each dataset in both cases

TABLE 10. The FPR of the developed method and other methods overall the tested datasets.

	Multi				Binary			
	KDDCup-99	NSL-KDD	BoT-IOT	CICIDS-2017	KDDCup-99	NSL-KDD	BoT-IOT	CICIDS-2017
PSO	0.02986	0.06063	0.00345	7.90E-05	0.07702	0.22747	7.40E-05	3.50E-05
MVO	0.0207	0.06112	0.00305	7.50E-05	0.07705	0.22374	8.00E-05	3.30E-05
GWO	0.0261	0.06108	0.00314	8.00E-05	0.07684	0.22716	8.00E-05	3.60E-05
MFO	0.0203	0.06086	0.00308	8.00E-05	0.07688	0.22614	7.20E-05	3.60E-05
WOA	0.02491	0.06103	0.00336	8.00E-05	0.0771	0.24264	8.00E-05	3.00E-05
FFA	0.02587	0.06061	0.00321	8.50E-05	0.07718	0.22813	8.00E-05	2.97E-05
BAT	0.0235	0.06131	0.00323	8.40E-05	0.07708	0.22937	7.50E-05	3.00E-05
TSODE	**0.01989**	**0.05868**	**0.00301**	**9.00E-05**	**0.07527**	**0.19223**	**7.00E-05**	**2.90E-05**
TSO	0.0224	0.06341	0.00339	8.70E-05	0.07708	0.22485	8.00E-05	5.00E-05

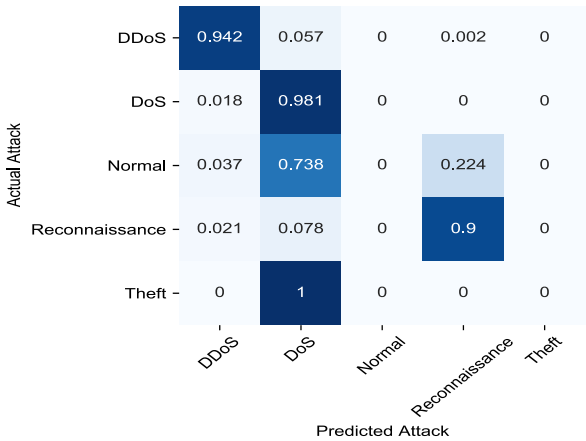


FIGURE 8. BoT-IoT dataset confusion matrix.

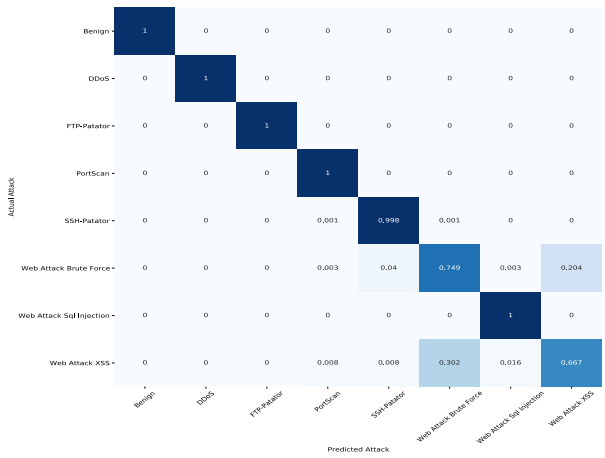


FIGURE 9. CICIDS-2017 dataset confusion matrix.

(i.e., binary and multi-classification). One can observed from these results that the developed TSODE has smallest CPU time(s) at three and two datasets in case of binary and multi-classification, respectively. In addition, the average of traditional TSO overall the tested datasets in both cases is better than other MH techniques.

TABLE 11. CPU time(s) for each algorithm among the tested dataset.

	Multi			
	Bot-IoT	KDDCup-99	NSL-KDD	CICIDS-2017
PSO	3.100	4.994	2462.677	162.035
MVO	3.010	4.185	2772.779	159.028
GWO	2.872	**4.053**	2848.010	158.322
MFO	3.347	5.266	2943.391	157.051
WOA	2.936	4.327	2538.937	157.397
FFA	2.847	4.489	2763.880	157.264
BAT	3.093	5.650	**2461.014**	172.718
TSODE	**2.600**	4.142	2518.707	**145.636**
TSO	2.710	4.874	2490.526	155.809
	Binary			
	Bot-IoT	KDDCup-99	NSL-KDD	CICIDS-2017
PSO	3.054	5.739	2661.675	175.835
MVO	2.913	4.393	**2274.733**	173.563
GWO	3.225	4.830	2393.259	173.054
MFO	3.138	4.787	2710.435	171.843
WOA	2.948	3.649	2451.573	172.471
FFA	2.935	4.489	2474.724	172.452
BAT	3.324	5.650	2621.755	189.920
TSODE	**2.728**	**3.432**	2395.802	**169.010**
TSO	3.047	5.051	2422.985	173.619

To further analyze the results, a non-parametric test named Friedman test is applied to determine if there is a significant difference between the developed method and others [59]. In this test, there are two hypothesises; the first hypothesis, called null hypothesises, which assumed there is no difference between the compared algorithms and we accept it when the P-value is greater than 0.05. Otherwise, we accept the second hypothesises, named alternative hypothesises, that assumed there is a significant difference between methods. Table 12 shows the mean rank of each method for the three tested datasets in two cases (i.e., binary and multi-classification). From those results, it can be noticed that in both cases of multi-classification, the developed TSODE has the highest mean rank in terms of all performance metrics. In addition, there is a significant difference between TSODE and other methods.

Moreover, the developed method is compared with the results of other methods collected from literature. We use

TABLE 12. Mean rank of each method using Friedman test in binary and multi classification.

	PSO	MVO	GWO	MFO	WOA	FFA	BAT	TSODE	TSO
Multi classification									
Accuracy	1	8	4.66	6	3	2	7	9	4.33
Recall	5	2.16	1	7	2.83	4	8	9	6
Precision	2.16	5.66	3.66	7.33	2.33	7.66	5.66	8.66	1.83
F1-Measure	1	3	7.33	7	4.33	2	6.5	8.66	5.16
CPU time(s)	6.25	5.75	4.75	7.25	4.75	4.50	6.50	2	3.25
Binary classification									
Accuracy	1	8	5.33	6.33	3	2	6	9	4.33
Recall	4.66	1.66	1.33	7	3	4.33	8	9	6
Precision	1.33	6	4.33	8	3	7	4.66	9	1.66
F1-Measure	1.66	2.66	7.66	6.33	4.33	3.33	6.33	9	3.66
CPU time(s)	7.75	3	5.25	5.75	3.75	4	8.25	1.50	5.75

KDDCup-99, as an example, in our comparison with BARF [60], Ref [61], Ref [62], Ref [63], Ref [64], and Ref [65] which have accuracy rates of 96.42, 95.21, 94.56, 93.36, 92.42, and 90.27, respectively.

VI. CONCLUSION

In this study, an intrusion detection system (IDS) for IoT systems was proposed using the advantages of deep learning and metaheuristic (MH) optimization algorithms. The developed system uses a Convolutional neural network (CNN) as a feature extractor technique to obtain relevant features from the input data. More so, we developed a new feature selection method using a new variant of the transient search optimization (TSO) algorithm using the deferential evolution (DE) algorithm. The DE operators are employed to boost the search process of the traditional TSD algorithm, as well as avoiding its shortcomings, such as trapping at local optima. We implemented extensive evaluation experiments to evaluate the developed method using three IoT IDS datasets, KDDCup-99, NSL-KDD, and BoT-IoT. Additionally, we compare the developed FS method, TSODE, to the traditional TSO and several well-known MH optimization methods. The outcome verified the prominent performance of the proposed method using different evaluation measures. We conclude that the proposed TSODE is significantly outperformed the traditional TSO since the application of the DE operators has improved the exploitation and exploration phases of the traditional TSO. More so, we conclude that the developed IDS scheme based on CNN and TSODE is significantly enhanced classification accuracy.

For future work, different MH optimizer will be considered for IDS with different datasets. Additionally, the performance of the TSODE makes a capability to use it in other optimization tasks, such as image processing, cloud and fog computing scheduling, parameter estimations, and others.

REFERENCES

[1] I. Lee, "The Internet of Things for enterprises: An ecosystem, architecture, and IoT service business model," *Internet Things*, vol. 7, Sep. 2019, Art. no. 100078.

[2] I. Lee, "Internet of Things (IoT) cybersecurity: Literature review and IoT cyber risk management," *Future Internet*, vol. 12, no. 9, p. 157, Sep. 2020.

[3] G. S. Kushwah and V. Ranga, "Voting extreme learning machine based distributed denial of service attack detection in cloud computing," *J. Inf. Secur. Appl.*, vol. 53, Aug. 2020, Art. no. 102532.

[4] P. Louvieris, N. Clewley, and X. Liu, "Effects-based feature identification for network intrusion detection," *Neurocomputing*, vol. 121, no. 18, pp. 265–273, 2013.

[5] P. Mishra, E. S. Pilli, V. Varadharajan, and U. Tupakula, "Intrusion detection techniques in cloud environment: A survey," *J. Netw. Comput. Appl.*, vol. 77, pp. 18–47, Jan. 2017.

[6] J. Wei, C. Long, J. Li, and J. Zhao, "An intrusion detection algorithm based on bag representation with ensemble support vector machine in cloud computing," *Concurrency Comput., Pract. Exper.*, vol. 32, no. 24, p. e5922, Dec. 2020.

[7] Q. Schueller, K. Basu, M. Younas, M. Patel, and F. Ball, "A hierarchical intrusion detection system using support vector machine for SDN network in cloud data center," in *Proc. 28th Int. Telecommun. Netw. Appl. Conf. (ITNAC)*, Nov. 2018, pp. 1–6.

[8] P. Ghosh, A. K. Mandal, and R. Kumar, "An efficient cloud network intrusion detection system," in *Information Systems Design and Intelligent Applications*. New Delhi, India: Springer, 2015, pp. 91–99.

[9] P. Deshpande, S. C. Sharma, S. K. Peddoju, and S. Junaid, "HIDS: A host based intrusion detection system for cloud computing environment," *Int. J. Syst. Assurance Eng. Manage.*, vol. 9, no. 3, pp. 567–576, Jun. 2018.

[10] C. Modi, D. Patel, B. Borisanya, A. Patel, and M. Rajarajan, "A novel framework for intrusion detection in cloud," in *Proc. 5th Int. Conf. Secur. Inf. Netw.*, 2012, pp. 67–74.

[11] K. Peng, V. C. M. Leung, L. Zheng, S. Wang, C. Huang, and T. Lin, "Intrusion detection system based on decision tree over big data in fog environment," *Wireless Commun. Mobile Comput.*, vol. 2018, Mar. 2018, Art. no. 4680867.

[12] X. Zhao and W. Zhang, "An anomaly intrusion detection method based on improved K-means of cloud computing," in *Proc. 6th Int. Conf. Instrum. Meas., Comput., Commun. Control (IMCCC)*, Jul. 2016, pp. 284–288.

[13] G. R. Kumar, N. Mangathayaru, and G. Narasimha, "An improved K-means Clustering algorithm for intrusion detection using Gaussian function," in *Proc. Int. Conf. Eng.*, 2015, pp. 1–7.

[14] C. Modi, D. Patel, B. Borisaniya, H. Patel, A. Patel, and M. Rajarajan, "A survey of intrusion detection techniques in cloud," *J. Netw. Comput. Appl.*, vol. 36, no. 1, pp. 42–57, 2013.

[15] K. A. P. da Costa, J. P. Papa, C. O. Lisboa, R. Munoz, and V. H. C. de Albuquerque, "Internet of Things: A survey on machine learning-based intrusion detection approaches," *Comput. Netw.*, vol. 151, pp. 147–157, Mar. 2019.

[16] H. Liu and B. Lang, "Machine learning and deep learning methods for intrusion detection systems: A survey," *Appl. Sci.*, vol. 9, no. 20, p. 4396, Oct. 2019.

[17] M. Almiani, A. AbuGhazleh, A. Al-Rahayfeh, S. Atiewi, and A. Razaque, "Deep recurrent neural network for IoT intrusion detection system," *Simul. Model. Pract. Theory*, vol. 101, May 2020, Art. no. 102031.

[18] E. Hodo, X. Bellekens, A. Hamilton, P.-L. Dubouilh, E. Iorkyase, C. Tachtatzis, and R. Atkinson, "Threat analysis of IoT networks using artificial neural network intrusion detection system," in *Proc. Int. Symp. Netw., Comput. Commun. (ISNCC)*, May 2016, pp. 1–6.

[19] A. Dawoud, S. Shahristani, and C. Raun, "Deep learning and software-defined networks: Towards secure IoT architecture," *Internet Things*, vols. 3–4, pp. 82–89, Oct. 2018.

[20] K. Wu, Z. Chen, and W. Li, "A novel intrusion detection model for a massive network using convolutional neural networks," *IEEE Access*, vol. 6, pp. 50850–50859, 2018.

[21] O. Alkadi, N. Moustafa, B. Turnbull, and K.-K.-R. Choo, "A deep blockchain framework-enabled collaborative intrusion detection for protecting IoT and cloud networks," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9463–9472, Jun. 2021.

[22] Z. Cai, Z. Xiong, H. Xu, P. Wang, W. Li, and Y. Pan, "Generative adversarial networks: A survey towards private and secure applications," 2021, *arXiv:2106.03785*. [Online]. Available: http://arxiv.org/abs/2106.03785

[23] M. T. Nguyen and K. Kim, "Genetic convolutional neural network for intrusion detection systems," *Future Gener. Comput. Syst.*, vol. 113, pp. 418–427, Dec. 2020.

[24] M. R. G. Raman, N. Somu, K. Kirthivasan, R. Liscano, and V. S. S. Sriram, "An efficient intrusion detection system based on hypergraph—Genetic algorithm for parameter optimization and feature selection in support vector machine," *Knowl.-Based Syst.*, vol. 134, pp. 1–12, Oct. 2017.

[25] S. Malhotra, V. Bali, and K. K. Paliwal, "Genetic programming and K-nearest neighbour classifier based intrusion detection model," in *Proc. 7th Int. Conf. Cloud Comput., Data Sci. Eng.*, Jan. 2017, pp. 42–46.

[26] P. Ghosh, A. Karmakar, J. Sharma, and S. Phadikar, "CS-PSO based intrusion detection system in cloud environment," in *Emerging Technologies in Data Mining and Information Security*. Singapore: Springer, 2019, pp. 261–269.

[27] J. K. Seth and S. Chandra, "MIDS: Metaheuristic based intrusion detection system for cloud using k-NN and MGWO," in *Proc. Int. Conf. Adv. Comput. Data Sci.* Singapore: Springer, Apr. 2018, pp. 411–420.

[28] S. P. RM, P. K. R. Maddikunta, M. Parimala, S. Koppu, T. R. Gadekallu, C. L. Chowdhary, and M. Alazab, "An effective feature engineering for DNN using hybrid PCA-GWO for intrusion detection in IoMT architecture," *Comput. Commun.*, vol. 160, pp. 139–149, Jul. 2020.

[29] M. Mayuranathan, M. Murugan, and V. Dhanakoti, "Best features based intrusion detection system by RBM model for detecting DDoS in cloud environment," *J. Ambient Intell. Hum. Comput.*, vol. 12, no. 3, pp. 3609–3619, 2019.

[30] R. SaiSindhuTheja and G. K. Shyam, "An efficient Metaheuristic algorithm based feature selection and recurrent neural network for DoS attack detection in cloud computing environment," *Appl. Soft Comput.*, vol. 100, Mar. 2021, Art. no. 106997.

[31] M. H. Qais, H. M. Hasanien, and S. Alghuwainem, "Transient search optimization: A new meta-heuristic optimization algorithm," *Int. J. Speech Technol.*, vol. 50, no. 11, pp. 3926–3941, Nov. 2020.

[32] W. Yang, K. Xia, T. Li, M. Xie, and Y. Zhao, "An improved transient search optimization with neighborhood dimensional learning for global optimization problems," *Symmetry*, vol. 13, no. 2, p. 244, Feb. 2021.

[33] J. S. Bhadoriya and A. R. Gupta, "A novel transient search optimization for optimal allocation of multiple distributed generator in the radial electrical distribution network," *Int. J. Emerg. Electr. Power Syst.*, 2021.

[34] R. Storn and K. Price, "Differential evolution—A simple and efficient heuristic for global optimization over continuous spaces," *J. Global Optim.*, vol. 11, no. 4, pp. 341–359, 1997.

[35] A. Shokuh Saljoughi, M. Mehrvarz, and H. Mirvaziri, "Attacks and intrusion detection in cloud computing using neural networks and particle swarm optimization algorithms," *Emerg. Sci. J.*, vol. 1, no. 4, pp. 179–191, Jan. 2018.

[36] S. Sharma, A. Gupta, and S. Agrawal, "An intrusion detection system for detecting denial-of-service attack in cloud using artificial bee colony," in *Proc. Int. Congr. Inf. Commun. Technol.* Singapore: Springer, 2016, pp. 137–145.

[37] T. Dash, "A study on intrusion detection using neural networks trained with evolutionary algorithms," *Soft Comput.*, vol. 21, no. 10, pp. 2687–2700, May 2017.

[38] A. Kannan, G. Q. Maguire, A. Sharma, and P. Schoo, "Genetic algorithm based feature selection algorithm for effective intrusion detection in cloud networks," in *Proc. IEEE 12th Int. Conf. Data Mining Workshops*, Dec. 2012, pp. 416–423.

[39] A. Nazir and R. A. Khan, "A novel combinatorial optimization based feature selection method for network intrusion detection," *Comput. Secur.*, vol. 102, Mar. 2021, Art. no. 102164.

[40] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165130–165150, 2020.

[41] K. Kumar, R. Kumar, T. de Boissiere, L. Gestin, W. Z. Teoh, J. Sotelo, A. de Brébisson, Y. Bengio, and A. C. Courville, "MelGAN: Generative adversarial networks for conditional waveform synthesis," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 14910–14921.

[42] A. Howard, M. Sandler, B. Chen, W. Wang, L.-C. Chen, M. Tan, G. Chu, V. Vasudevan, Y. Zhu, R. Pang, H. Adam, and Q. Le, "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1314–1324.

[43] J. Angel, S. T. Aroyehun, A. Tamayo, and A. Gelbukh, "NLP-CIC at SemEval-2020 Task 9: Analysing sentiment in code-switching language using a simple deep-learning classifier," in *Proc. 14th Workshop Semantic Eval.*, Barcelona, Spain, Dec. 2020, pp. 957–962. [Online]. Available: https://www.aclweb.org/anthology/2020.semeval-1.123

[44] S. Merity, "Single headed attention RNN: Stop thinking with your head," 2019, *arXiv:1911.11423*. [Online]. Available: http://arxiv.org/abs/1911.11423

[45] A. Ororbia, A. ElSaid, and T. Desell, "Investigating recurrent neural network memory structures using neuro-evolution," in *Proc. Genetic Evol. Comput. Conf.*, Jul. 2019, pp. 446–455.

[46] A. Bochkovskiy, C.-Y. Wang, and H.-Y. Mark Liao, "YOLOv4: Optimal speed and accuracy of object detection," 2020, *arXiv:2004.10934*. [Online]. Available: http://arxiv.org/abs/2004.10934

[47] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27th Int. Conf. Mach. Learn. (ICML)*, 2010, pp. 807–814.

[48] B. McFee, J. Salamon, and J. P. Bello, "Adaptive pooling operators for weakly labeled sound event detection," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 26, no. 11, pp. 2180–2193, Nov. 2018.

[49] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*. [Online]. Available: http://arxiv.org/abs/1412.6980

[50] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE ICNN*, vol. 4, Nov./Dec. 1995, pp. 1942–1948.

[51] S. Mirjalili, S. M. Mirjalili, and A. Hatamlou, "Multi-verse optimizer: A nature-inspired algorithm for global optimization," *Neural Comput. Appl.*, vol. 27, no. 2, pp. 495–513, 2016.

[52] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey wolf optimizer," *Adv. Eng. Softw.*, vol. 69, pp. 46–61, Mar. 2014.

[53] S. Mirjalili, "Moth-flame optimization algorithm: A novel nature-inspired heuristic paradigm," *Knowl.-Based Syst.*, vol. 89, pp. 228–249, Nov. 2015.

[54] S. Mirjalili and A. Lewis, "The whale optimization algorithm," *Adv. Eng. Softw.*, vol. 95, pp. 51–67, May 2016.

[55] X.-S. Yang and X. He, "Firefly algorithm: Recent advances and applications," *Int. J. Swarm Intell.*, vol. 1, no. 1, pp. 36–50, 2013.

[56] X. S. Yang, "A new metaheuristic bat-inspired algorithm," in *Nature Inspired Cooperative Strategies for Optimization*. Berlin, Germany: Springer, 2010, pp. 65–74.

[57] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Gener. Comput. Syst.*, vol. 100, pp. 779–796, Nov. 2019.

[58] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, vol. 1, Jan. 2018, pp. 108–116.

[59] M. Friedman, "A comparison of alternative tests of significance for the problem of m rankings," *Ann. Math. Statist.*, vol. 11, no. 1, pp. 86–92, 1940.

[60] J. Li, Z. Zhao, R. Li, H. Zhang, and T. Zhang, "Ai-based two-stage intrusion detection for software defined IoT networks," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2093–2102, Nov. 2018.

[61] J. Li, Z. Zhao, and R. Li, "A machine learning based intrusion detection system for software defined 5G network," 2017, *arXiv:1708.04571*. [Online]. Available: http://arxiv.org/abs/1708.04571

[62] A. S. da Silva, J. A. Wickboldt, L. Z. Granville, and A. Schaeffer-Filho, "ATLANTIC: A framework for anomaly traffic detection, classification, and mitigation in SDN," in *Proc. IEEE/IFIP Netw. Oper. Manage. Symp.*, Apr. 2016, pp. 27–35.

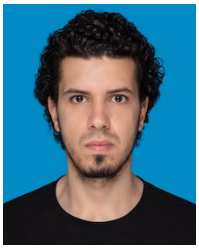
[63] X. Ye, X. Chen, H. Wang, X. Zeng, and G. Shao, "An anomalous behavior detection model in cloud computing," *Tsinghua Sci. Technol.*, vol. 21, no. 3, pp. 322–332, Jun. 2016.

[64] P. Wang, K.-M. Chao, H.-C. Lin, W.-H. Lin, and C.-C. Lo, "An efficient flow control approach for SDN-based network threat detection and migration using support vector machine," in *Proc. IEEE 13th Int. Conf. E-Bus. Eng. (ICEBE)*, Nov. 2016, pp. 56–63.

[65] A. Le, P. Dinh, H. Le, and N. C. Tran, "Flexible network-based intrusion detection and prevention system on software-defined networks," in *Proc. Int. Conf. Adv. Comput. Appl. (ACOMP)*, Nov. 2015, pp. 106–111.

ABDULAZIZ FATANI received the B.S. degree in computer sciences from Umm Alqura University, Makkah, Saudi Arabia, in 2009, and the M.S. degree in computer sciences from Huazhong University of Science and Technology, Wuhan, China, in 2015, where he is currently pursuing the Ph.D. degree in computer sciences. In 2010, he worked at Umm Alqura University.

MOHAMED ABD ELAZIZ received the B.S. and M.S. degrees in computer science and the Ph.D. degree in mathematics and computer science from Zagazig University, Egypt, in 2008, 2011, and 2014, respectively. From 2008 to 2011, he was an Assistant Lecturer with the Department of Computer Science. He is currently an Associate Professor with Zagazig University. He is the author of more than 190 articles. He is one of the 2% influential scholars, which depicts the 100,000 top-scientists in the world. His research interests include metaheuristic technique, security IoT, cloud computing, machine learning, signal processing, image processing, and evolutionary algorithms.



ABDELGHANI DAHOU received the B.S. and M.S. degrees in computer science and intelligent systems from the University of Ahmad Draia, Adrar, Algeria, in 2012 and 2014, respectively, and the Ph.D. degree in computer science from Wuhan University of Technology, Wuhan, Hubei, China, in 2019. He is currently a Lecturer with the Faculty of Science and Technology, University of Ahmad Draia. His research interests include deep learning, signal processing, data mining, neuro-evolution, image processing, and natural language processing.

MOHAMMED A. A. AL-QANESS received the B.S., M.S., and Ph.D. degrees from Wuhan University of Technology, in 2010, 2014, and 2017, respectively, all in information and communication engineering. He is currently an Assistant Professor with the School of Computer Science, Wuhan University, Wuhan, China. He is also a Postdoctoral Follower with the State Key Laboratory for Information Engineering in Surveying, Mapping, and Remote Sensing, Wuhan University. His current research interests include wireless sensing, mobile computing, machine learning, signal and image processing, and natural language processing.

SONGFENG LU was born in 1968. He received the Ph.D. degree in computer software and theory from Huazhong University of Science and Technology. He is currently a Professor of Huazhong University of Science and Technology. His research interests include artificial intelligence, quantum computing, and information security.